

Get to Know Pointers

CSC 230 : C and Software Tools

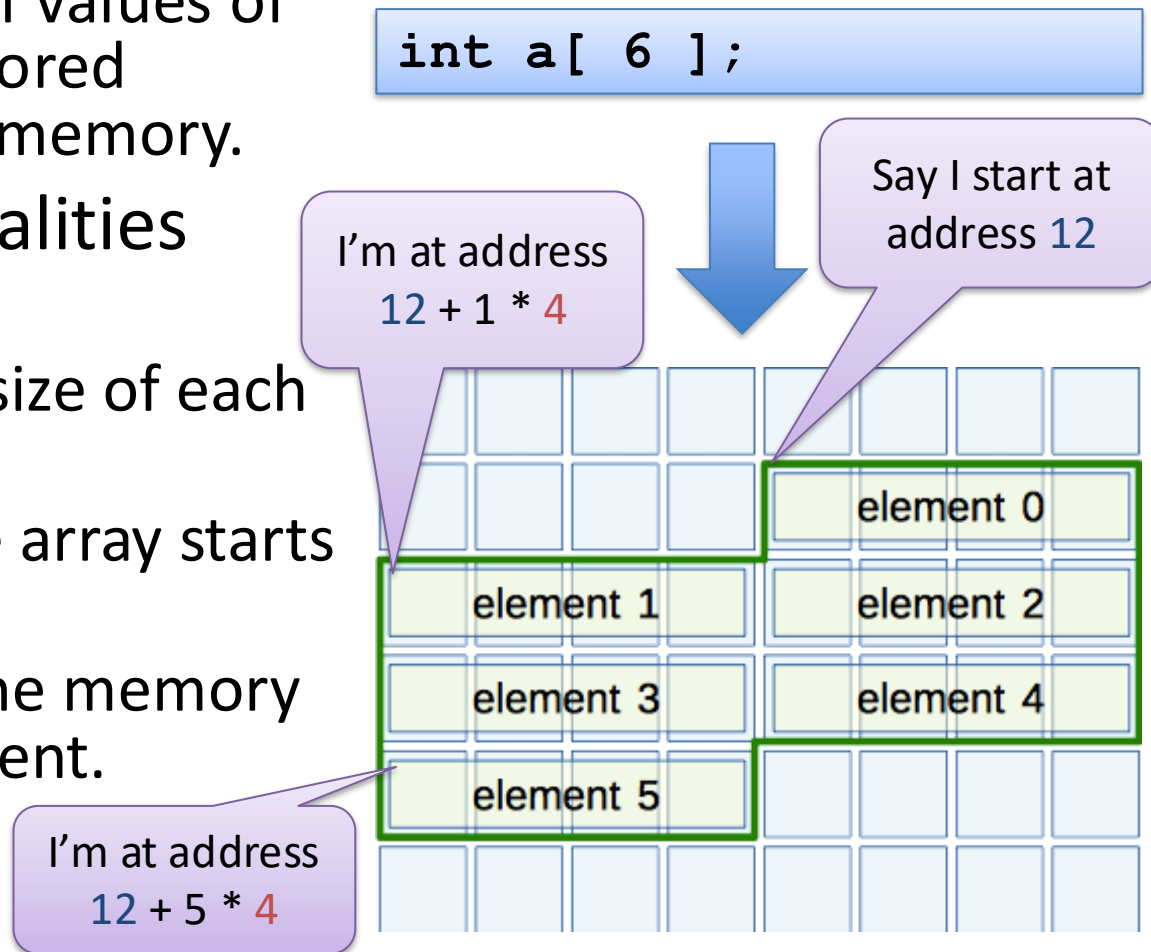
NC State Department of Computer
Science

Topics for Today

- Pointers and arrays
- Pointer arithmetic
- Command-line arguments
- Void pointers
- Block I/O

Array Internals

- Really, what is an array?
 - It's a sequence of values of the **same type** stored **consecutively** in memory.
- Why do these qualities matter?
 - If you know the size of each element
 - ... and where the array starts in memory
 - ... you can find the memory storing any element.



Array Internals

- So, to use an array, the compiler needs to keep up with:
 - How big each element is, so it can do indexing
 - That's implied by the **element type**
 - The location of the first element in memory
 - That sounds like **a pointer**
- And, the compiler will give you this pointer if you want it.

Arrays as Pointers

- We have a general rule.
 - If **a** is an **array of some type T**
a evaluates to a **pointer to the first element**
with a **type of pointer to T** (i.e., **T ***)

```
int a[ 6 ];  
int *ap = a;
```

a evaluates to pointer to int.

```
float b[ 100 ];  
float *bp = b;
```

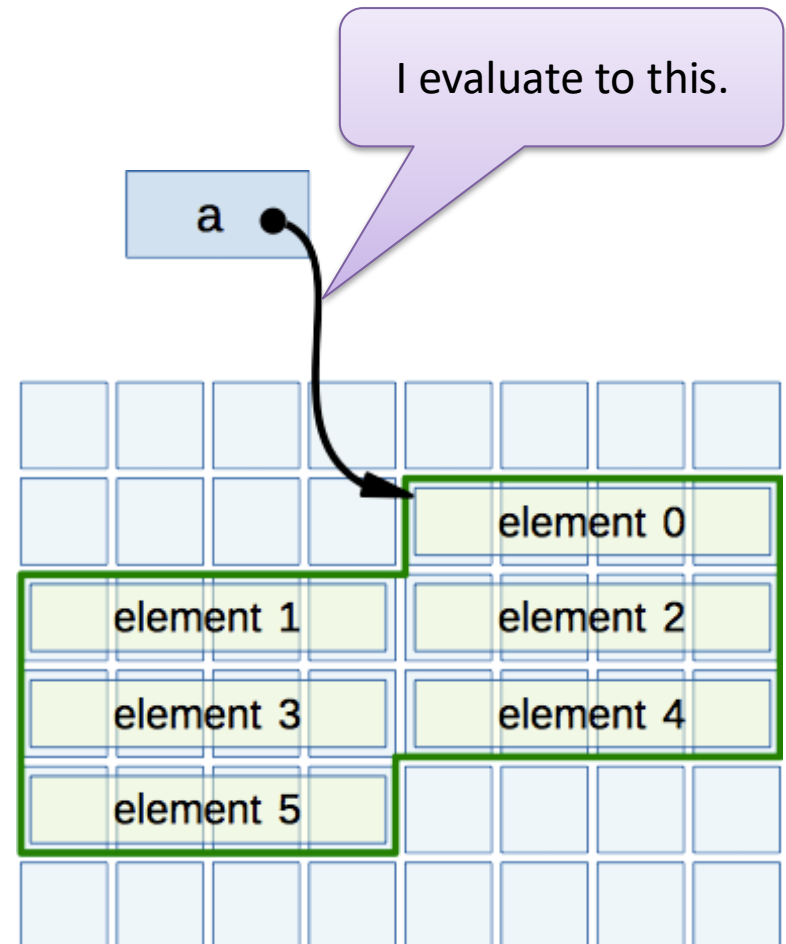
b evaluates to pointer to float.

```
Whatever c[ 10 ];  
Whatever *cp = c;
```

c evaluates to pointer to
Whatever.

Arrays as Pointers ... Illustrated!

```
int a[ 6 ];
```

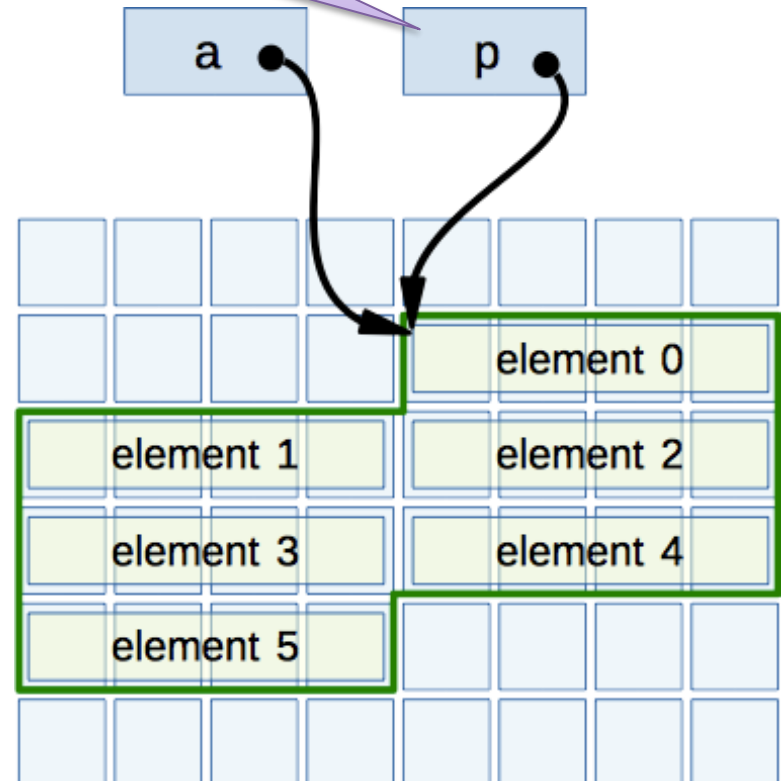


Arrays as Pointers ... Illustrated!

```
int a[ 6 ];
```

```
int *p = a;
```

Now I do too.



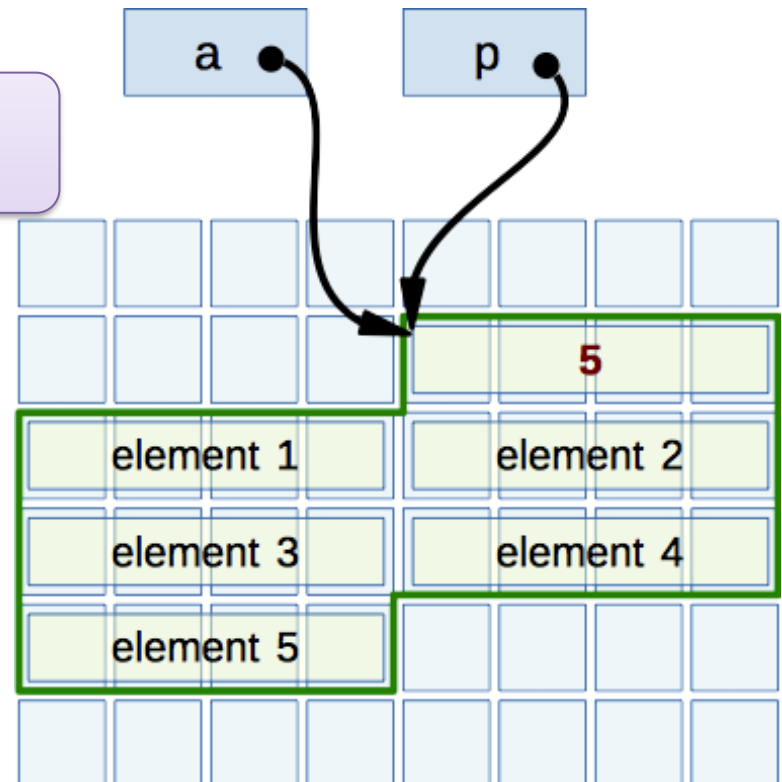
Arrays as Pointers ... Illustrated!

```
int a[ 6 ];
```

```
int *p = a;
```

```
a[ 0 ] = 5;
```

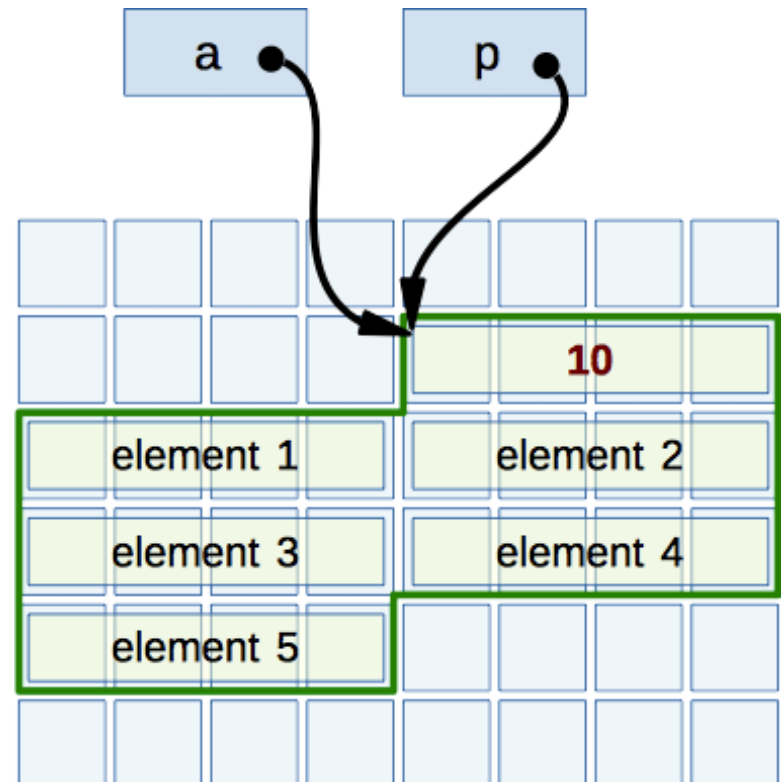
I can change the first element.



Arrays as Pointers ... Illustrated!

```
int a[ 6 ];  
  
int *p = a;  
  
a[ 0 ] = 5;  
  
*p = 10;
```

Me too.

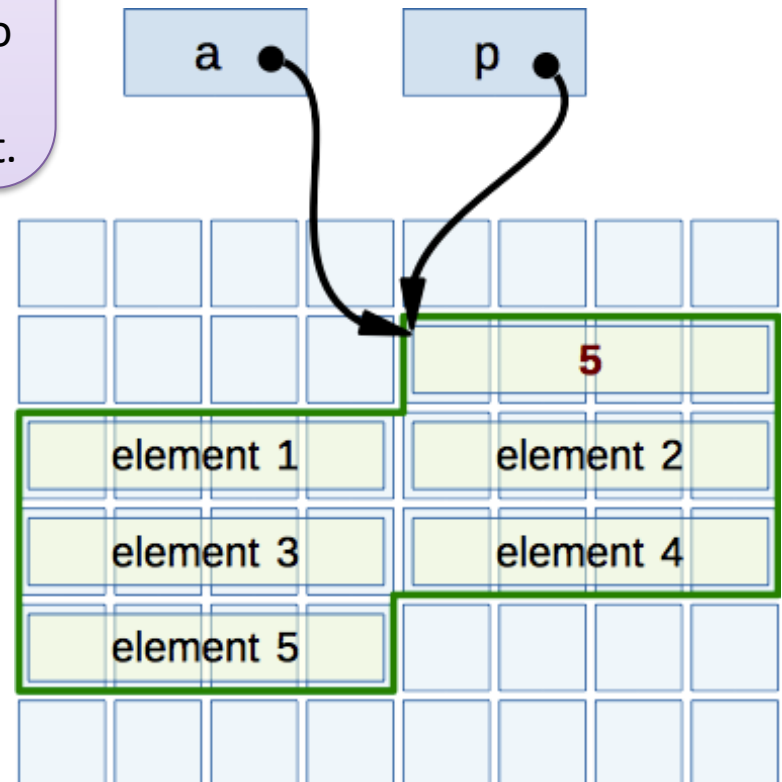


Arrays as Pointers ... Illustrated!

```
int a[ 6 ];  
  
int *p = a;  
  
a[ 0 ] = 5;  
  
*p = 10;  
  
*a = 5;
```

Legal?

Sure. The array name evaluates to a pointer, so we can dereference it.

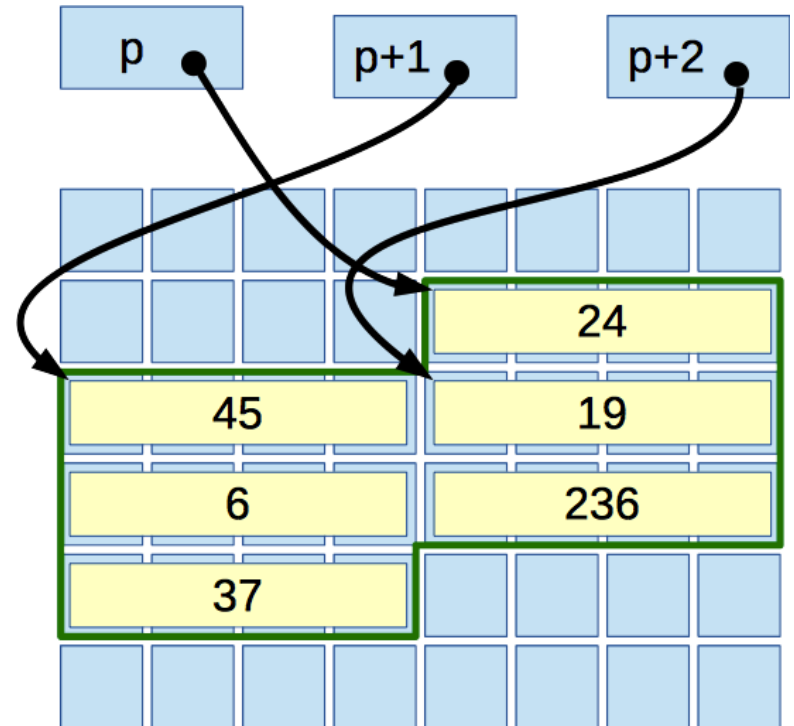


Arrays as Pointers

- Useful? Well no, not very useful ... yet.
- This just gives us two ways of getting to the first array element.
- But, what if we could move this pointer around in memory?
 - Maybe we could get to other elements of the array.

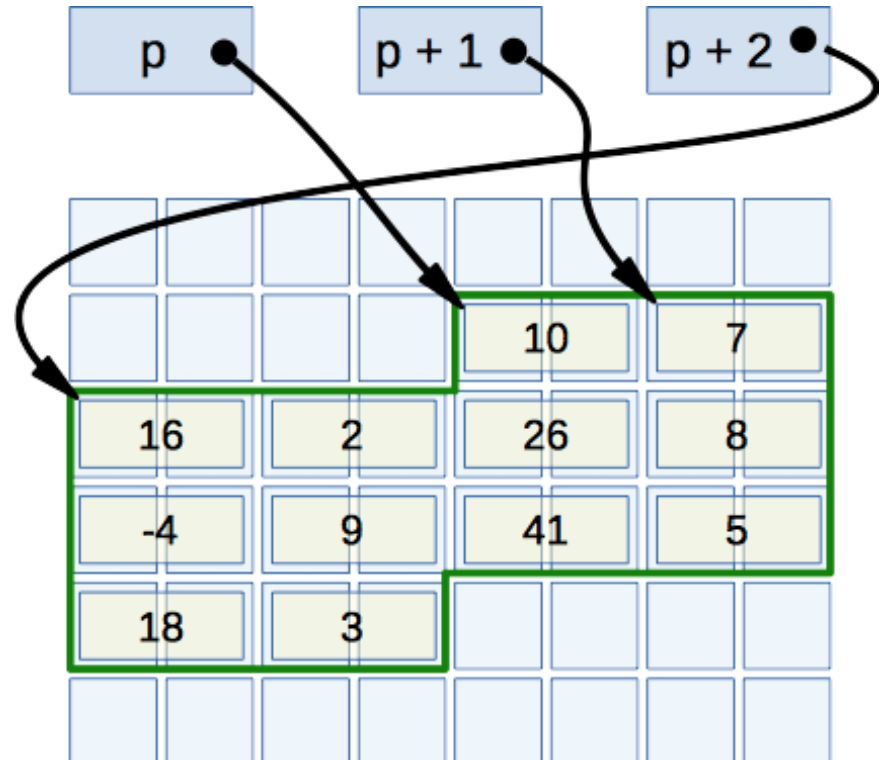
Pointer Arithmetic

- We can do (some) math on pointers, and it behaves in a very useful way.
 - It moves by **multiples of sizeof** for the type of object pointed to.
 - Adding 1 to an int pointer moves ahead 4 bytes.
 - Adding 2 to an int pointer moves ahead 8 bytes.
 - Adding 3 to an int pointer ... well, you get the idea.



Pointer Arithmetic

- This works for any type
 - Adding to a short pointer moves by 2 bytes at a time.
 - Adding to a long pointer moves by 8 bytes at a time.
 - Adding to a char pointer moves one byte at a time.
 - ...



Pointer Arithmetic

- We can see the effects of pointer arithmetic if we print the pointer's value.

```
char ca[ 10 ];  
int ia[ 10 ];  
double da[ 10 ];
```

```
char *cp = ca;  
int *ip = ia;  
double *dp = da;
```

```
printf( "%p %p %p\n", cp, ip, dp );  
printf( "%p %p %p\n", cp + 1, ip + 1, dp + 1 );
```

I'm 1 byte later.

I'm 4 bytes later.

I'm 8 bytes later.

Array Elements via Pointers

- This behavior makes it easy to access nearby elements of the same type.

```
int a[ 10 ];  
int *p = a;
```

Pointer dereference to
access the first element.

```
*p = 25;  
*( p + 1 ) = 35;  
*( p + 2 ) = 75;
```

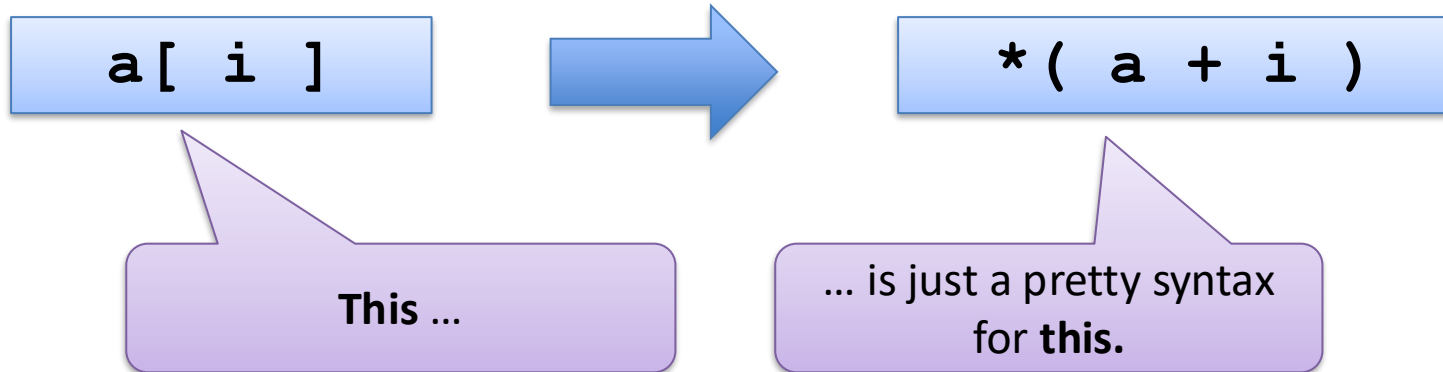
Pointer arithmetic and
dereference to access other
elements.

```
*a = 26;  
*( a + 1 ) = 36;  
*( a + 2 ) = 76;
```

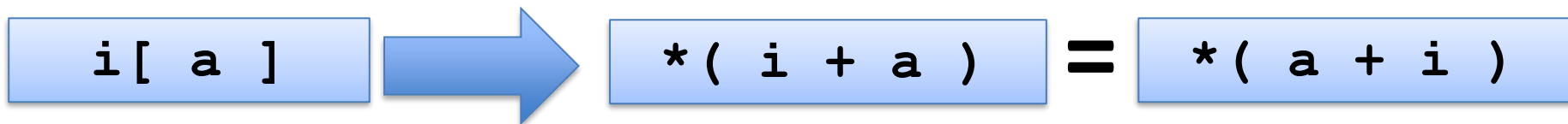
Array name evaluates to a
pointer, so you can even do
this. In fact, you're in for a
surprise ...

Array Indexing Explained

- This is what array indexing means in C



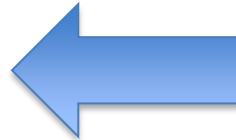
- Addition is commutative, so this is why we can write indexing backward:



Indexing With a Pointer

- If $a[i]$ is just a nice way of writing $*(a+i)$...
- ... and $*(p+i)$ works for any pointer p ... then:

$*(p + i)$



$p[i]$

We already know this works for a pointer

... and we can write it like this, since it means the same thing anyway.

- Arrays and pointers **aren't** the same thing,
 - But, in lots of situations, we can use them interchangeably.

Passing Arrays as Pointers

- So, this gives us an alternative way to pass arrays to functions (or you might think so)
 - Just pass the address of the first element.

```
void g( int *p )  
{  
    p[ 0 ] += 1;  
    p[ 1 ] += 2;  
}
```

I'm a pointer to the first element.

You can index from me, like I was an array.

```
int myList[ ] = { 1, 4, 9 };  
g( myList );
```

I evaluate to the address of the first element.

Array Parameters Explained

- But, this is secretly how we've been passing arrays all along.
 - We've just been using an alternative syntax.

```
void g( int p[] )  
{  
    p[ 0 ] += 1;  
    p[ 1 ] += 2;  
}
```

When you ask
for this ...

```
void g( int *p )  
{  
    p[ 0 ] += 1;  
    p[ 1 ] += 2;  
}
```

... the compiler
does this.

```
int myList[ ] = { 1, 4, 9 };  
g( myList );
```

```
int myList[ ] = { 1, 4, 9 };  
g( myList );
```

You've Been Lied To

- So far, we've described string-handling functions as taking a char array parameter:

```
size_t strlen(char const str[])
```

```
int sprintf(char str[], char const format[], ...);
```

```
strcpy(char dest[], char const src[])
```

- Really, you will probably see them documented as char pointer:

```
size_t strlen(char const *str)
```

```
int sprintf(char *str, char const *format, ...);
```

```
strcpy(char *dest, char const *src)
```

Passing an Array Offset

- Array Parameters are passed by pointer
- Any pointer will do, it doesn't have to be at the start of an array.

```
void capitalize( char *msg )  
{  
    for ( int i = 0; msg[i]; i++ ) {  
        msg[i] = toupper( msg[ i ] );  
    }  
}
```

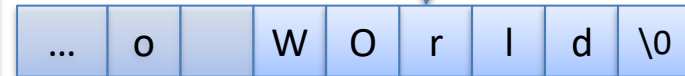
I'll take any char pointer.

I can still use it like an array.

msg

```
char str[] = "hello world";  
capitalize( str + 6 );
```

Here, just work on the last part of this array.



Marking Your Spot

- `scanf()` and `fscanf()` automatically move forward in the input as you read.
- But not `sscanf()`, it just reads the string you give it from the start.

```
sscanf( buffer, "%d", &value );  
sscanf( buffer, "%s", word );  
Sscanf( buffer, "%s", word2 );
```

This will start
back at the start
of buffer.

21 Jump Street

Marking Your Spot

- We can fix this with a little pointer arithmetic.
- `sscanf()` can tell you how many characters you've read so far.

I don't count in the return value from `scanf()`

```
sscanf( buffer, "%d%n", &value, &pos );
```

- With pointer arithmetic, we can use this to get a pointer later into the same array.

```
sscanf( buffer + pos, "%s", word );
```

Arrays Aren't Pointers

- You might start to think arrays and pointers are the same thing ... but you'd be wrong.
- We can see a few situations where they don't behave exactly the same:

```
int a[ 6 ];  
int *p = a;
```

```
... sizeof( a )
```

```
int b[ 6 ];  
a = b;
```

I'm the size of the whole array.

I'm just the size of a pointer.

You can't change where an array is stored.

But, you can make a pointer point elsewhere.

```
int a[ 6 ];  
int *p = a;
```

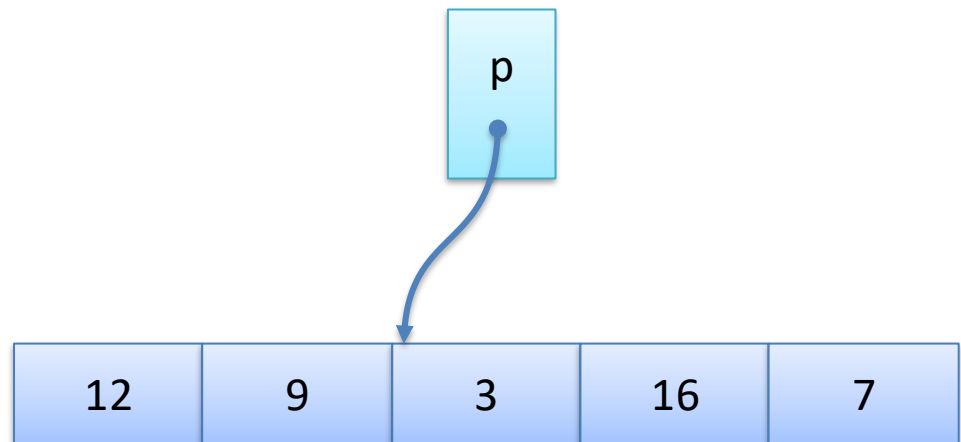
```
... sizeof( p ) ...
```

```
int b[ 6 ];  
p = b;
```

Pointer Arithmetic

- We can use ++ and -- operators with pointers, to move forward or backward by one element.
 - Often combined with dereferencing.
 - Consider, what will each of these do?

<code>p++;</code>
<code>--p;</code>
<code>*--p = 5;</code>
<code>--*p;</code>
<code>*p++ = 5;</code>
<code>(*p)++;</code>
<code>--*--p;</code>



Pointer Arithmetic

```
p++;
```

```
--p;
```

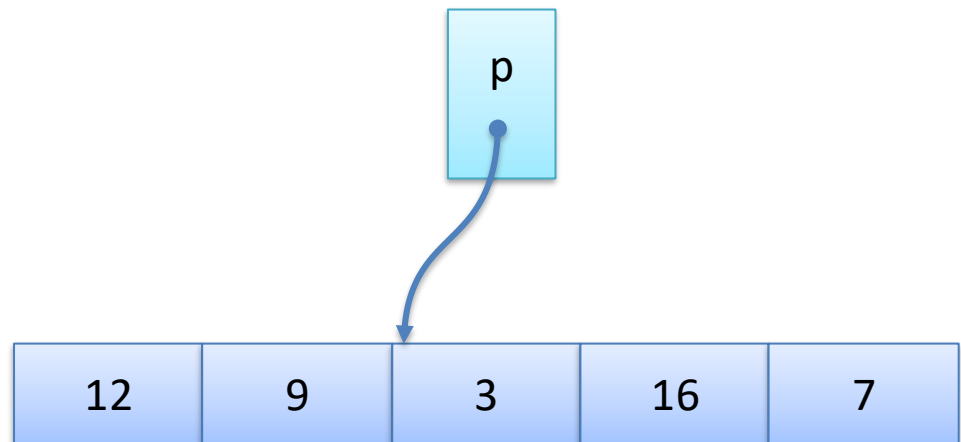
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

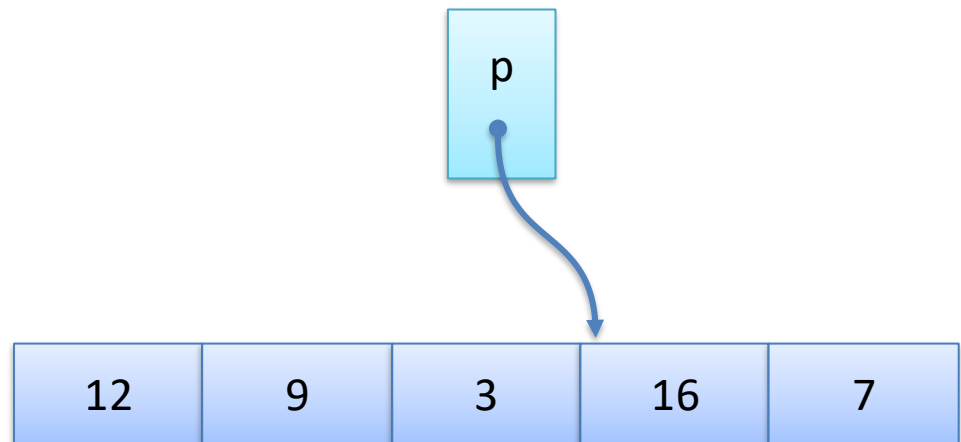
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

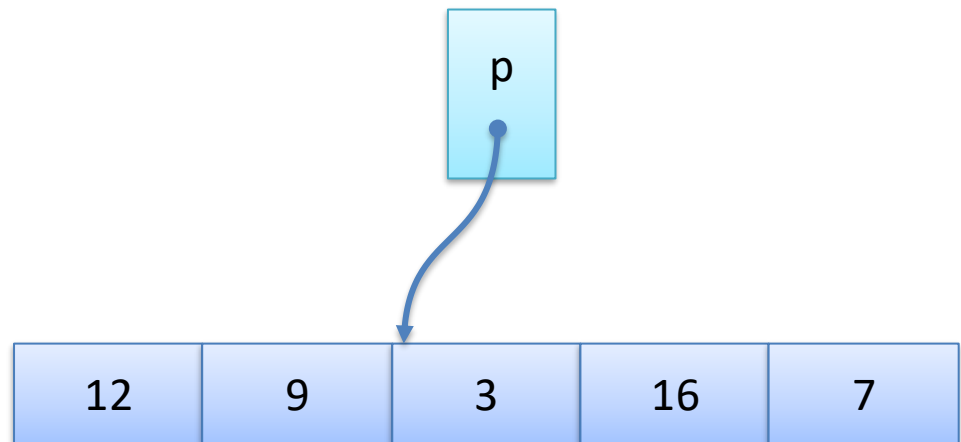
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

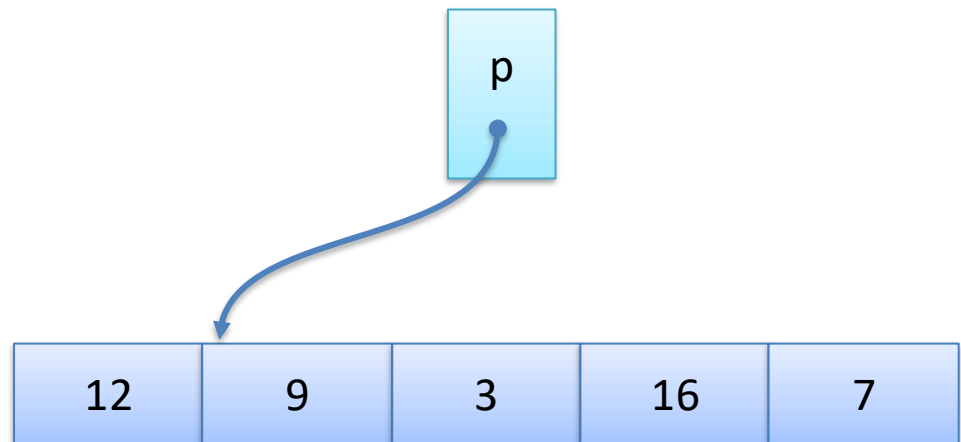
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

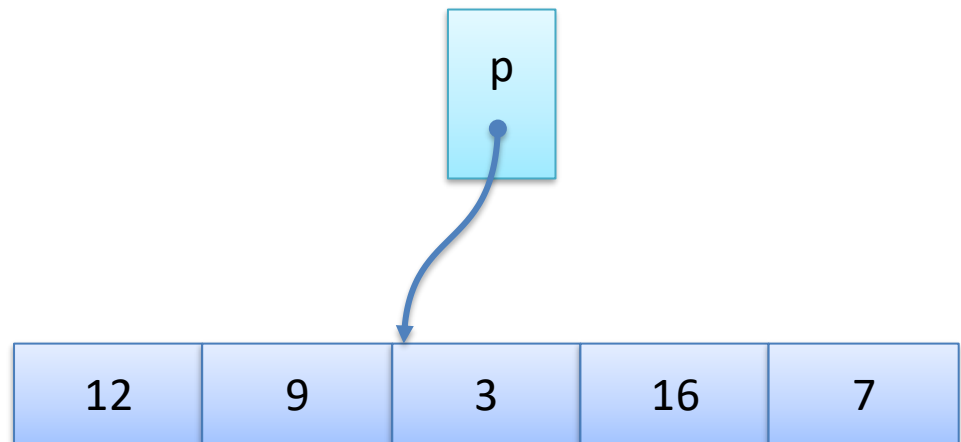
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

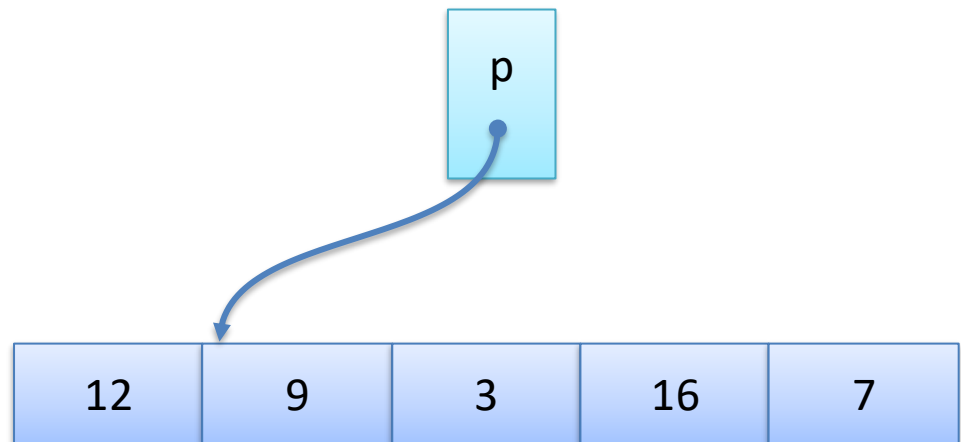
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

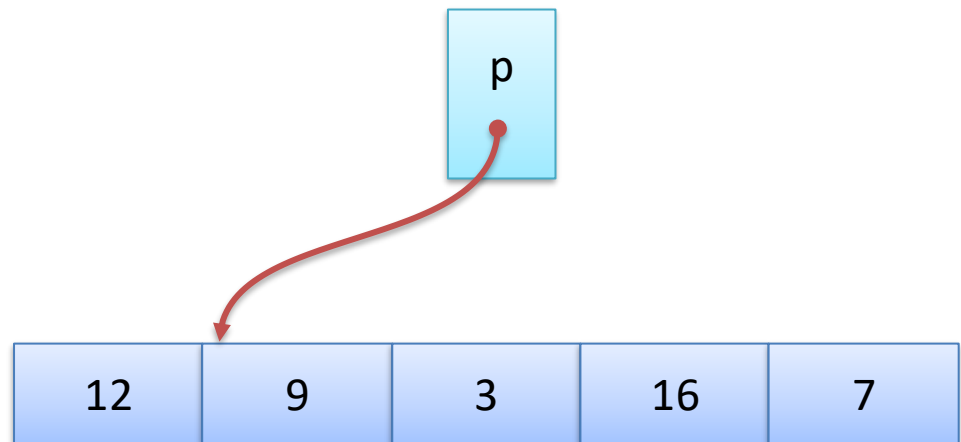
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

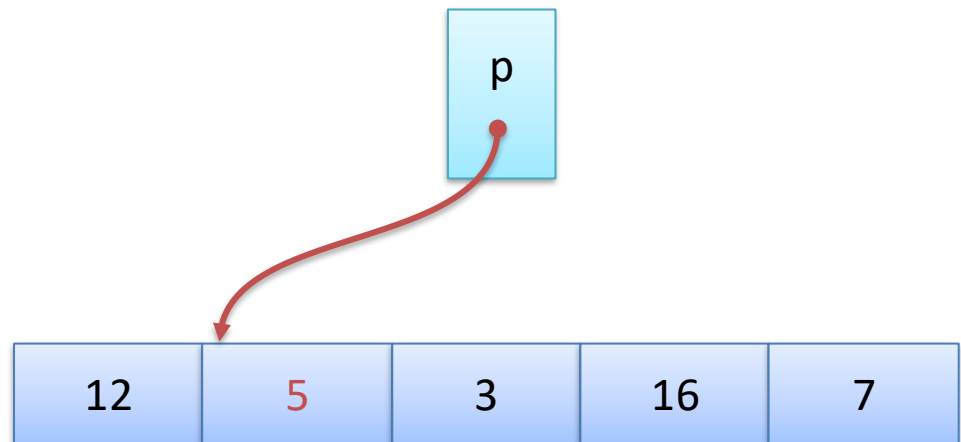
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

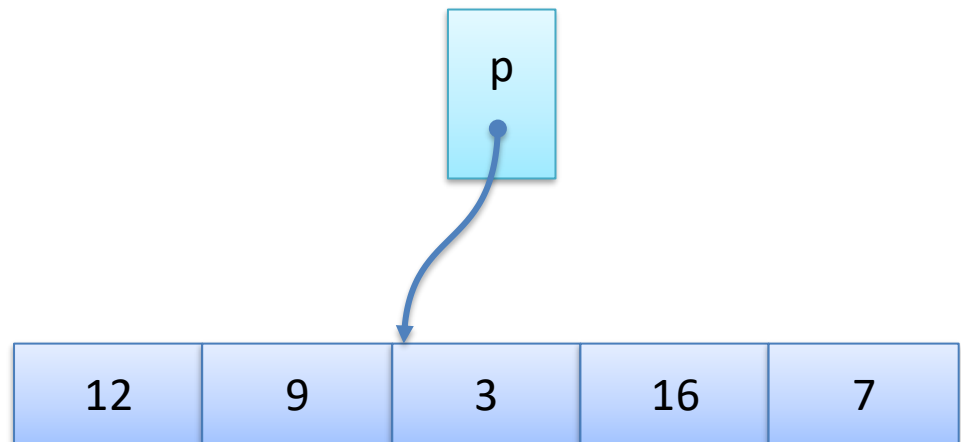
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

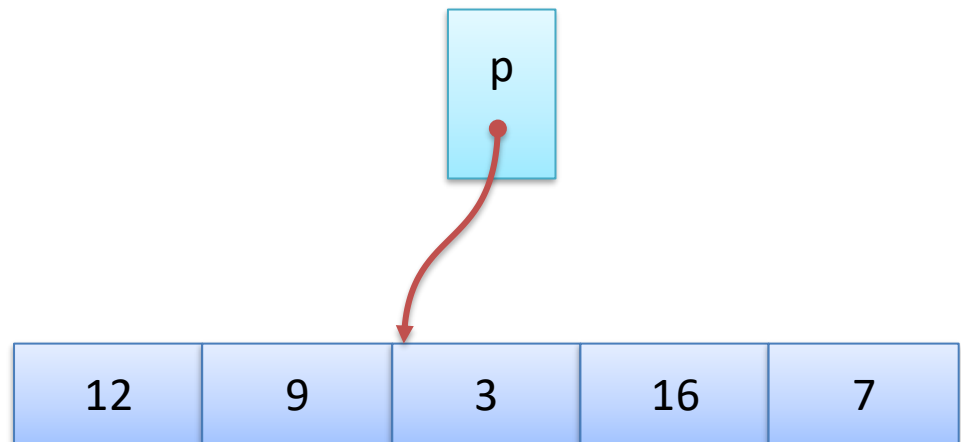
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

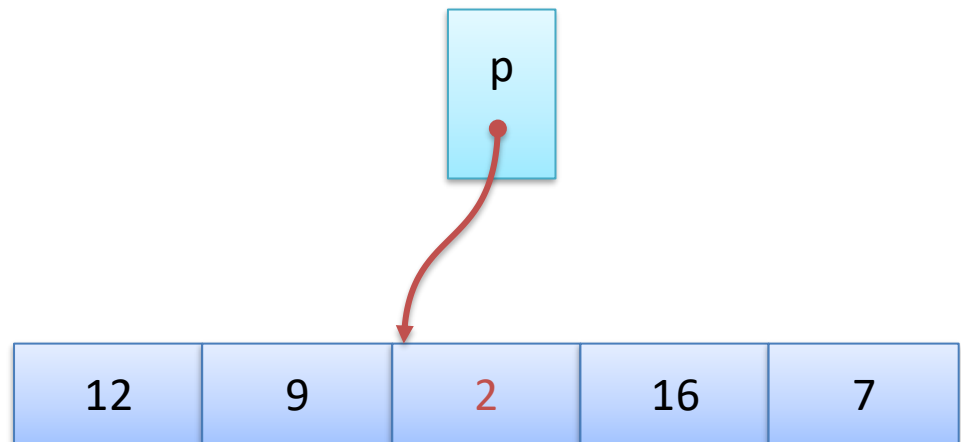
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

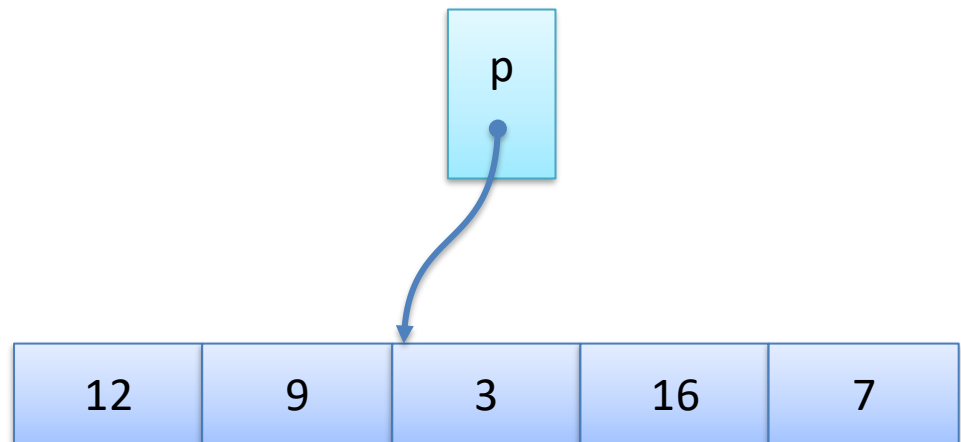
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

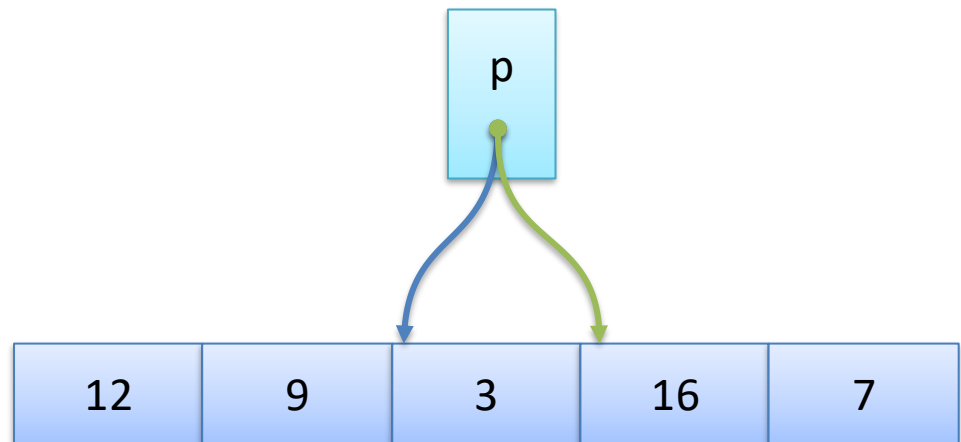
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

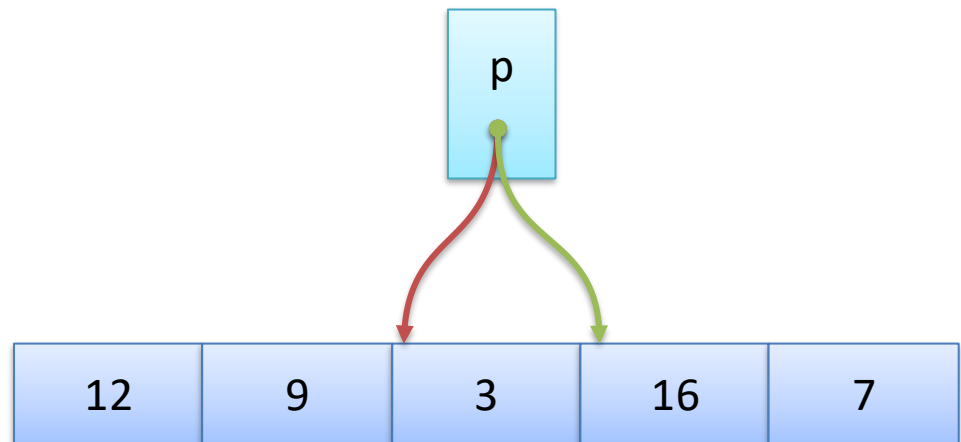
```
*--p = 5;
```

```
--*p;
```

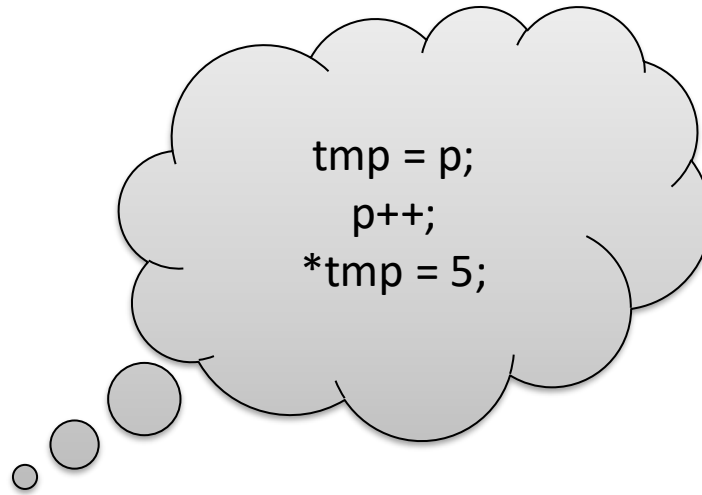
```
*p++ = 5;
```

```
(*p)++;
```

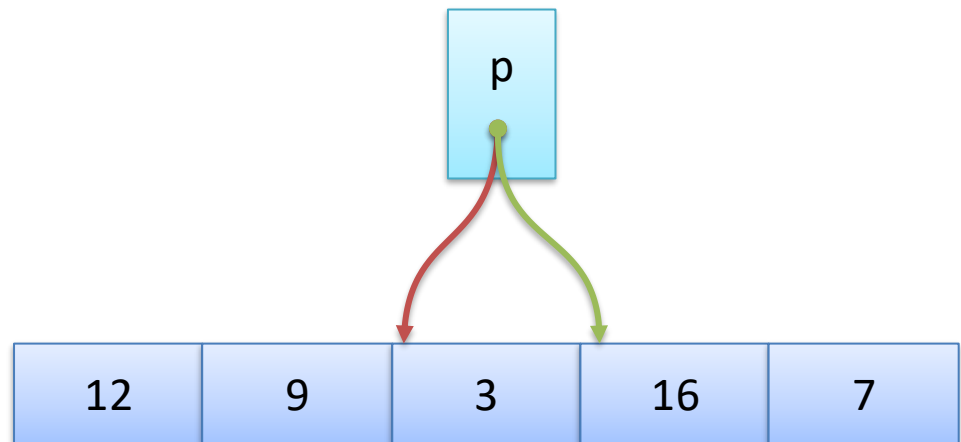
```
--*--p;
```



Pointer Arithmetic

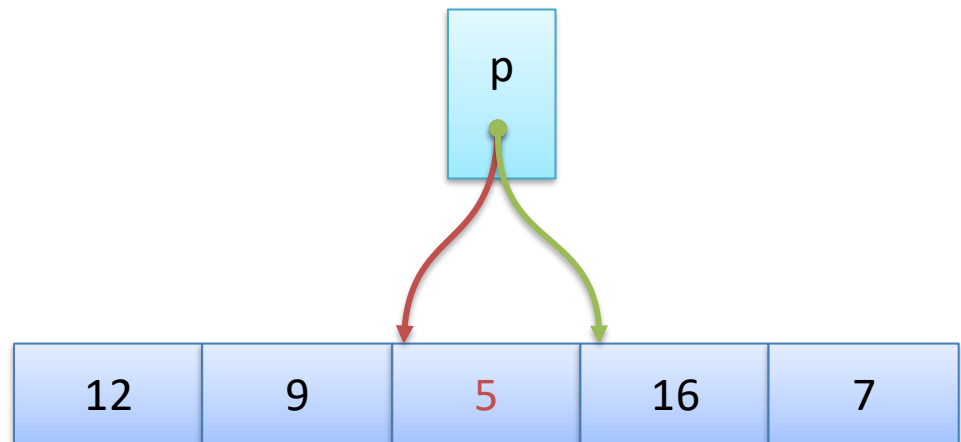


p++;
--p;
*--p = 5;
--*p;
*p++ = 5;
(*p)++;
--*--p;



Pointer Arithmetic

<code>p++;</code>
<code>--p;</code>
<code>*--p = 5;</code>
<code>--*p;</code>
<code>*p++ = 5;</code>
<code>(*p)++;</code>
<code>--*--p;</code>



Pointer Arithmetic

```
p++;
```

```
--p;
```

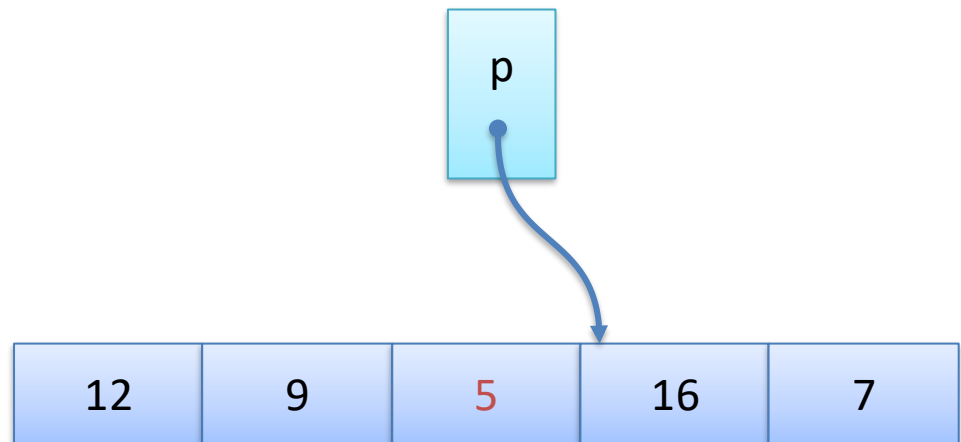
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

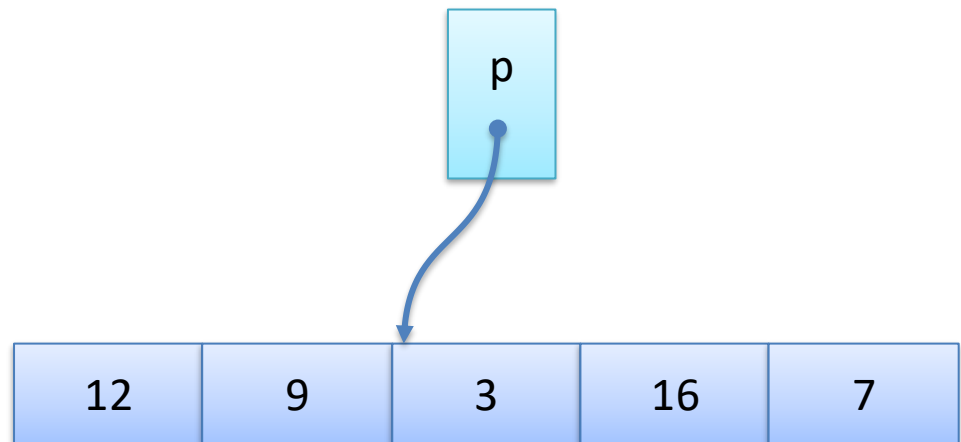
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

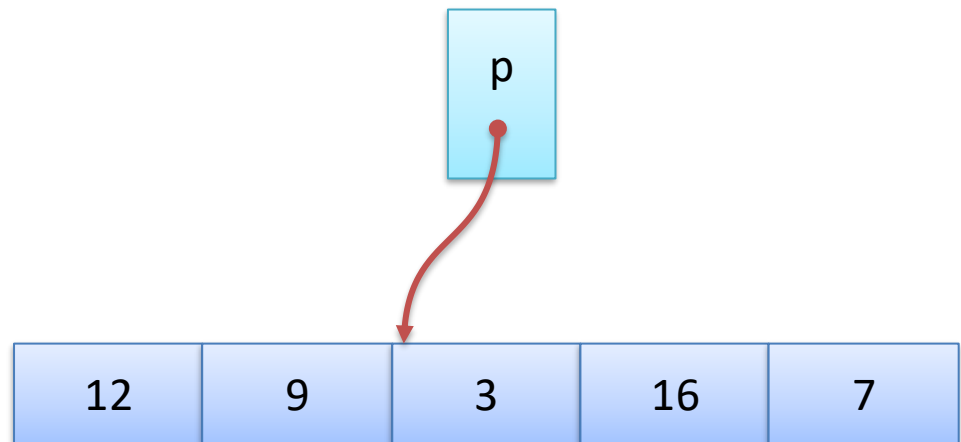
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

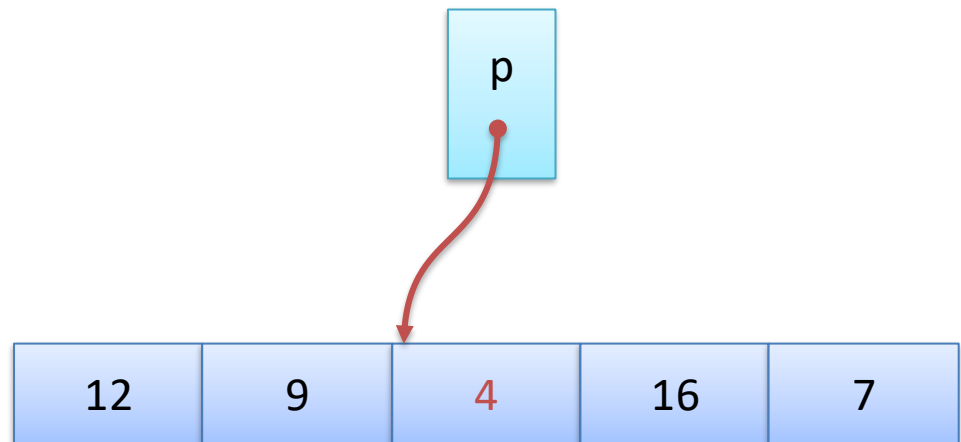
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

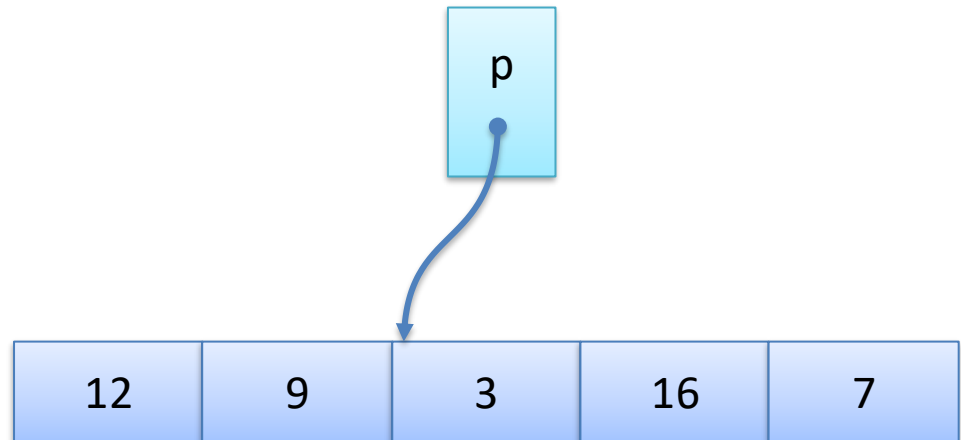
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

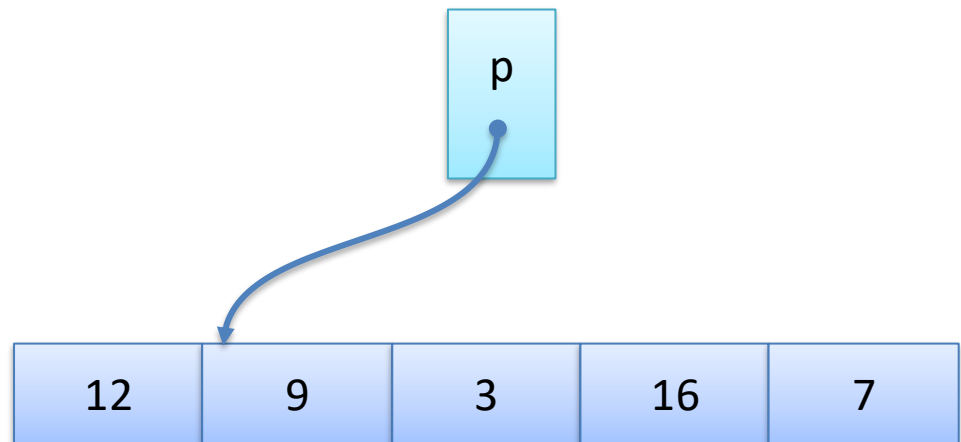
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

```
p++;
```

```
--p;
```

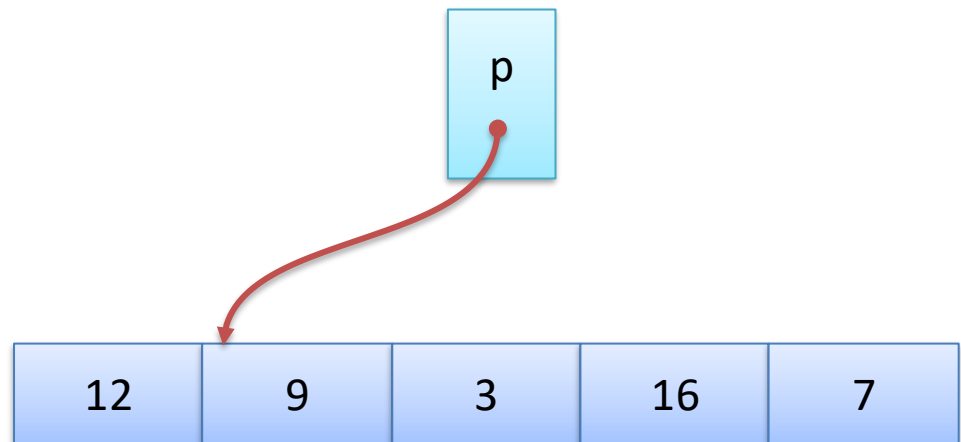
```
*--p = 5;
```

```
--*p;
```

```
*p++ = 5;
```

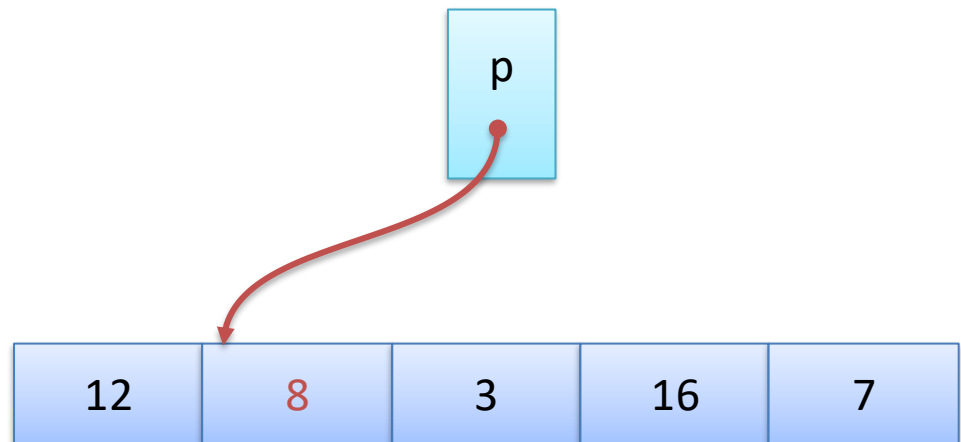
```
(*p)++;
```

```
--*--p;
```



Pointer Arithmetic

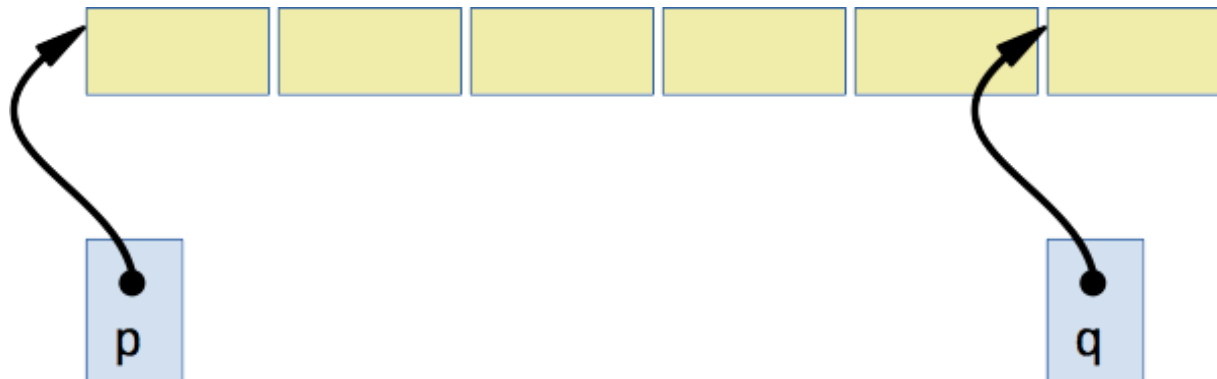
<code>p++;</code>
<code>--p;</code>
<code>*--p = 5;</code>
<code>--*p;</code>
<code>*p++ = 5;</code>
<code>(*p)++;</code>
<code>--*--p;</code>



Useful Pointer Arithmetic

- You can add and subtract pointers and numbers

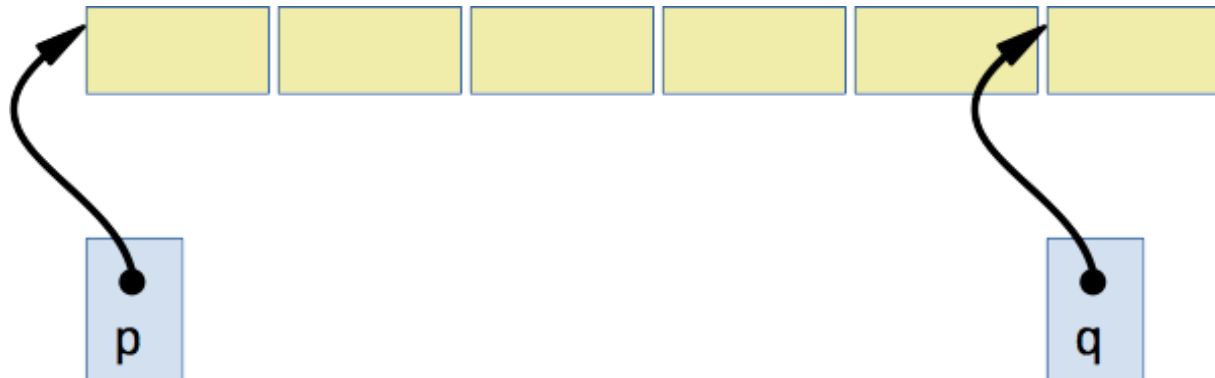
pointer + number $\rightarrow$ pointer	These each evaluate to a pointer to a nearby value.
number + pointer $\rightarrow$ pointer	
pointer - number $\rightarrow$ pointer	
pointer - pointer $\rightarrow$ number	This evaluates to the number of values between two nearby pointers.
✗ number - pointer $\rightarrow$ nonsense	Not all operations are meaningful.



Useful Pointer Arithmetic

- You can compare pointers

<code>pointer == pointer</code>	<code>pointer != pointer</code>
<code>pointer < pointer</code>	<code>pointer > pointer</code>
<code>pointer <= pointer</code>	<code>pointer >= pointer</code>



Alternatives in Array Indexing

- We have some choices in how we can process and iterate over arrays.
- Plain-old array syntax:

```
int a[] = { 1, 4, 9, 16, 25 };  
int len = sizeof( a ) / sizeof( a[ 0 ] );
```

```
int sum = 0;  
for ( int i = 0; i < len; i++ )  
    sum += a[ i ];
```

Alternatives in Array Indexing

- Or, with a pointer visiting each element.

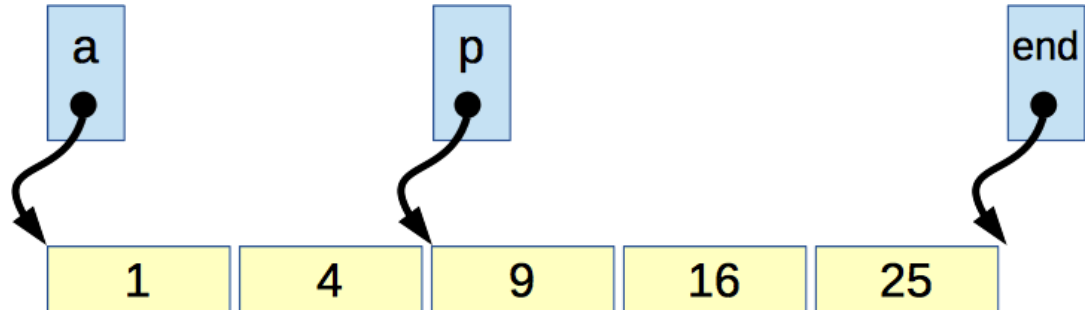
```
int a[] = { 1, 4, 9, 16, 25 };  
int len = sizeof( a ) / sizeof( a[ 0 ] );
```

```
int *end = a + len;  
  
int sum = 0;  
for ( int *p = a; p < end; p++ )  
    sum += *p;
```

Pointer right past the end of the array. This is OK, as long as we don't dereference this pointer.

Iterate from the start of the array, stopping when we pass its last element.

Less to do here.
This could give you
a little performance
improvement.

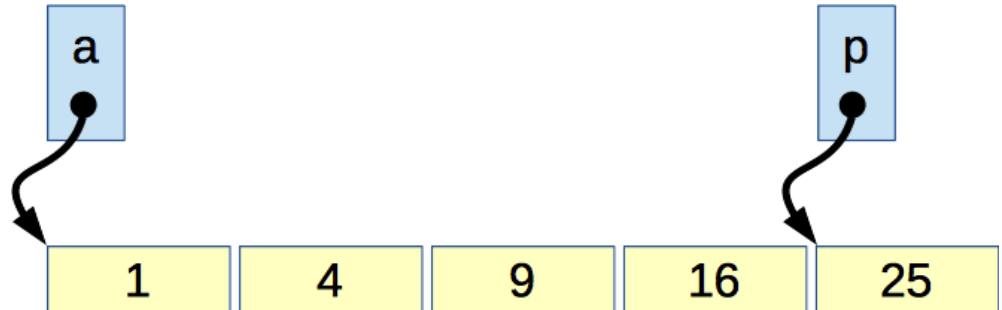


Alternatives in Array Indexing

- Or, we can work backward, from the end toward the start.

```
int a[] = { 1, 4, 9, 16, 25 };  
int len = sizeof( a ) / sizeof( a[ 0 ] );
```

```
for ( int *p = a + len - 1; p >= a; p-- )  
    sum += *p;
```



Alternatives in Array Indexing

- One more example, measuring string length:

```
int strlen1( char const *str )
{
    int i = 0;
    while ( str[ i ] != '\0' )
        i++;

    return i;
}
```

Here's the interesting part.
The difference in pointers
tells us how far away p got
from the start.

```
int strlen2( char const *str )
{
    char const *p = str;
    while ( *p )
        p++;

    return p - str;
}
```

Literal Strings

- Literal strings are a special syntax for making an array.
- The constant `"abc123"` evaluates to a char pointer.
- When the compiler sees a string literal like `"abc123"`
 - It allocates an array for the string literal (with a null character at the end)
 - And uses the address of this array as the value of the literal
- You can even evaluate things like:
 - `"abc123"[ 2 ]`

String Creation Alternatives

- This gives us some options in how to make a string variable:

```
// Make me a new array containing this.  
char str1[] = "abc123";
```

```
// Make this string and just give a pointer  
char *str2 = "xyz789";
```

- This allocates and initializes memory differently:

str1:

a	b	c	1	2	3	\0
---	---	---	---	---	---	----

Initialized every time the declaration executed. Modifiable.

str2:

•

 →

x	y	z	7	8	9	\0
---	---	---	---	---	---	----

Allocated statically, generally not modifiable.

Command Line Arguments

- You can run a C program with command-line arguments:

```
shell$ ./myProgram red green "pale blue"
```

You need quotes to put a special character in an argument.

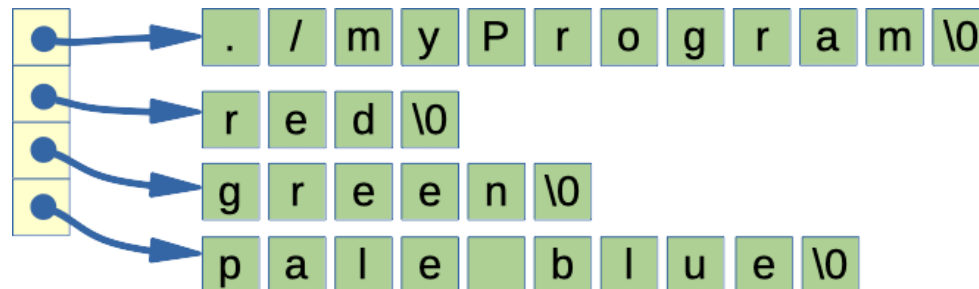
I'm argument 0

I'm argument 1

I'm argument 2

I'm argument 3

- These are stored as an array of char pointers, each one pointing to one of the arguments:



Command Line Arguments

- To see these arguments, you can define main with two parameters:

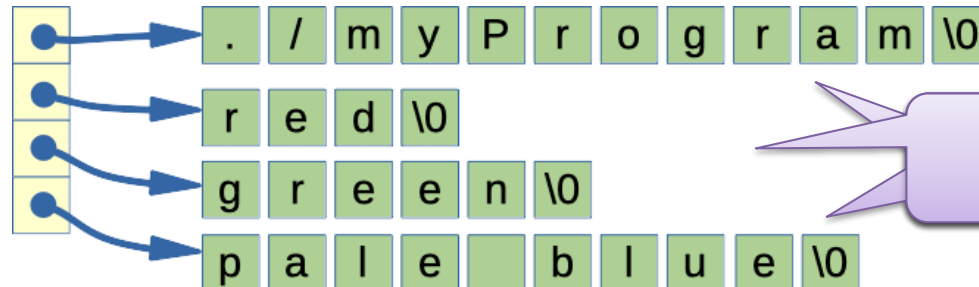
```
int main( int argc, char *argv[] ) ...
```

Number of arguments.

Array of char pointers, one for each argument

argv is this array of pointers

argc is its length.



We're each strings.

Command Line Arguments

- Say, we wanted to just report the arguments:

```
int main( int argc, char *argv[] )
{
    printf( "%d arguments\n", argc );

    for ( int i = 0; i < argc; i++ )
        printf( "    %s\n", argv[ i ] );
}
```

- More typically, we would use the arguments to change how the program operates.

Void Pointers

- You can't have a value of type void

```
 void v;
```

- But you can make a pointer to void

```
void *vPtr;
```

- And that void pointer can hold any pointer value you want

```
float f = 3.14;  
vPtr = &f;  
  
long l = 123456789;  
vPtr = &l;
```

You don't even
need a cast.

Void Pointers

- But, you can't directly dereference a void pointer.

```
printf( "%ld\n", *vPtr );
```

- Think of a void pointer as a generic way of holding an address
 - they could be used to point to anything
 - but, to use what they point to, you need to know its type.

Using Void Pointers

- You need to convert a void * to a more specific type before you dereference.

```
long *lPtr = vPtr;  
printf( "%ld\n", *lPtr );
```

You don't even need a cast.

```
lPtr = (long *)vPtr;  
printf( "%ld\n", *lPtr );
```

But, it's probably a good idea.

- Void pointers are used:
 - As parameter/return types for functions that can work with memory (no matter what it stores)
 - As temporary holders for pointers of various types.

Block Read/Write Functions

- Reading and writing individual bytes can be slow
- C gives us functions to read/write **arrays of bytes**
 - If you're not reading/writing characters → you probably want a binary stream.

Any type of array to write from.

Size of each element.

```
size_t fwrite(const void *ptr, size_t size,  
              size_t nmemb, FILE *stream);
```

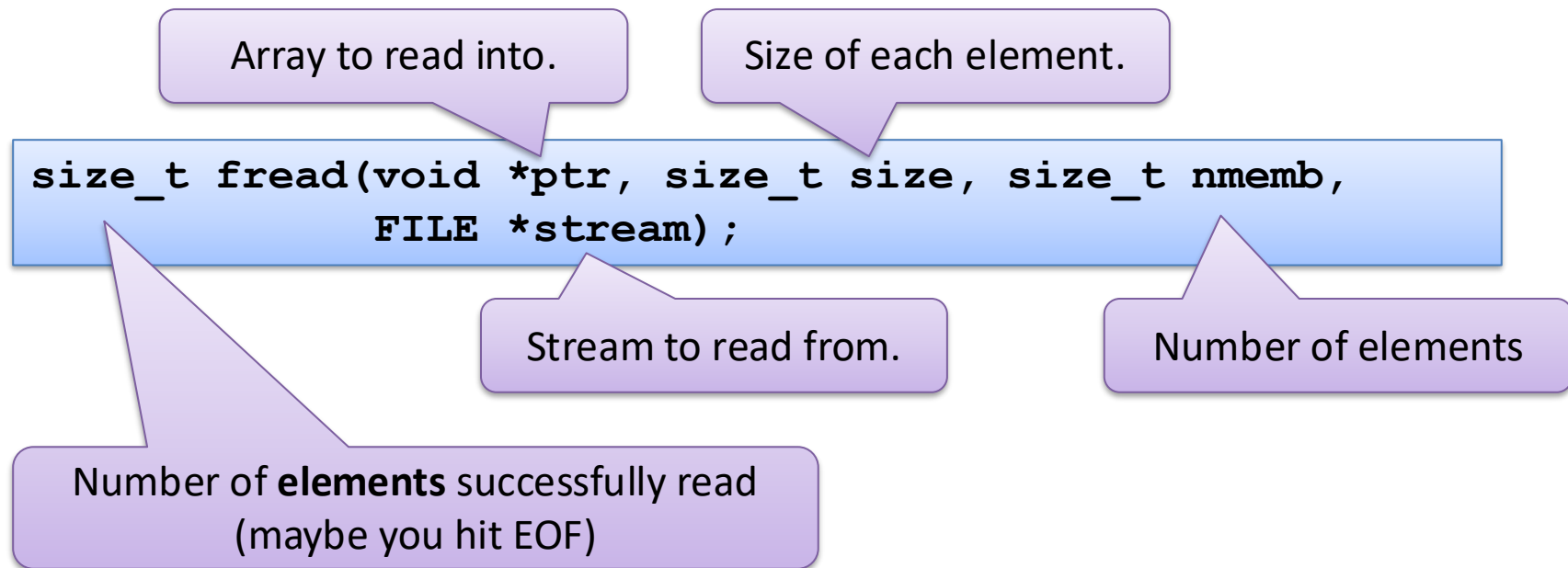
Number of elements

Stream to write to.

Number of elements successfully written.

Block Read/Write Functions

- We have a similar function for reading.



- These are a lot like the underlying calls provided by the OS

Block I/O Example

- Here, we're copying a file up to 1024 bytes at a time.

```
FILE *src = fopen( argv[ 1 ], "rb" );  
FILE *dest = fopen( argv[ 2 ], "wb" );  
  
char buffer[ 1024 ];  
int len;  
while ( ( len = fread( buffer, 1, sizeof(buffer),  
                      src ) ) != 0 )  
    fwrite( buffer, 1, len, dest );  
  
fclose( src );  
fclose( dest );
```

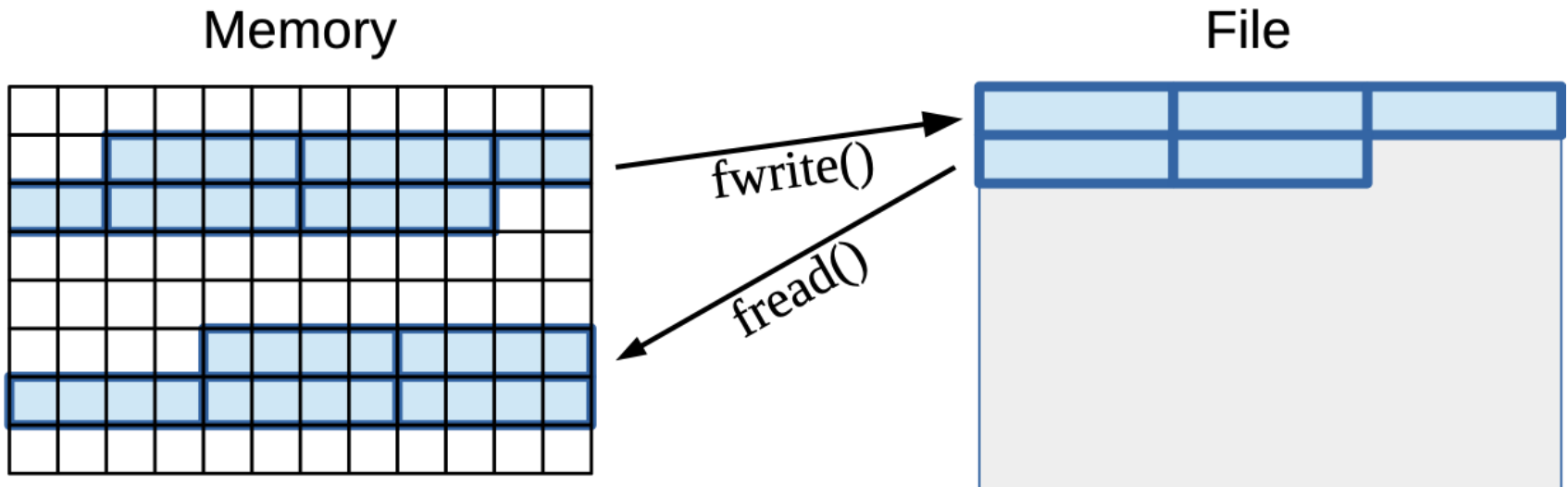
Open two files named on the command line, in binary mode.

Read blocks of up to 1024 bytes.

Write however many bytes we just read.

fread() and fwrite() Illustrated

- You can think of fread() and fwrite() like this.
 - They copy arrays of data between memory and files.



Block I/O Example

- We can serialize/deserialize a whole array at a time:

```
FILE *fp = fopen( "list.bin", "wb" );  
  
int list[] = { 3, 472, 13729, 257381, 719293945 };  
  
fwrite( list, sizeof(int), 5, fp );  
fclose( fp );
```

Write the contents of this array in memory.

```
FILE *fp = fopen( "list.bin", "rb" );  
  
int list[ 5 ];  
  
fread( list, sizeof(int), 5, fp );  
fclose( fp );
```

Read it Back.

Binary Representation and Byte Order

- We can look at the contents of this file
 - The hexdump command can help
 - We can see where the individual bits are stored (on the common platform)
 - $472_{10} = 000001D8_{16}$

```
$ hexdump -C list.bin
00000000  03 00 00 00 d8 01 00 00  a1 35 00 00 65 ed
00000010  f9 8d df 2a
```

Bits 0 .. 7

Bits 8 .. 15

Bits 16 .. 23

Bits 24 .. 31

Binary Representation and Byte Order

```
$ hexdump -C list.bin
00000000  03 00 00 00 d8 01 00 00  a1 35 00 00 65 ed
00000010  f9 8d df 2a
```

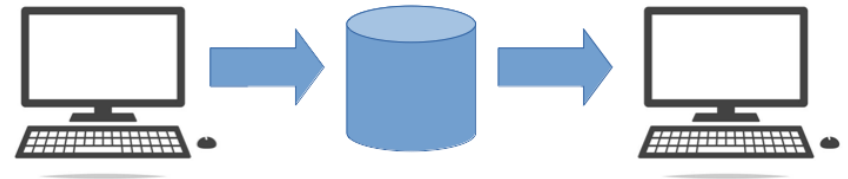
Low-order byte

High-order byte

- Here, we can see that the common platform is a *LSB-First* (Little-Endian) system
- But, other systems might arrange bytes in the opposite order, *MSB-First* (Big-Endian)

Worrying about Byte Order

- Differences in byte order can cause problems if we move binary files between platforms.
 - Reading and writing binary files.



- Network communication.



- It's generally the programmer's job to worry about this.

Binary I/O and Byte Order

- Of course, we can write code to reorder bytes:

```
void swapBytes4( int *val )
{
    char *p = (char *)val;

    char t = p[ 0 ];
    p[ 0 ] = p[ 3 ];
    p[ 3 ] = t;

    t = p[ 1 ];
    p[ 1 ] = p[ 2 ];
    p[ 2 ] = t;
}
```

Get a pointer to the bytes in this value.

Swap the first and last.

Swap the middle two.

- There are library functions to do this:

- Like htonl() and ntohl()

Host-to-network long
and
Network-to-host long